Microservices with Spring Cloud

Introduction

Microservices and RESTful APIs with Spring Boot and Spring Cloud - Preview

Congratulations

Introduction to the Course & Course Guide

Introduction to Web Services

What is a Web Service?

Important How Questions Related to Web Services

Web Services - Key Terminology

Introduction to SOAP Web Services

Introduction to RESTful Web Services

SOAP vs RESTful Web Services

Restful Web Services with Spring Boot

Section Introduction - RESTful Web Services with Spring Boot

Step 01 - Initializing a RESTful Services Project with Spring Boot

Fastest Approach to Solve All Your Exceptions

Step 02 - Understanding the RESTful Services We Would Construct in this Course

Step 03 - Constructing a Hello World Service

Step 04 - Enhancing the Hello World Service to Return a Bean

Step 06 - Enhancing the Hello World Service with a Path Variable

Step 07 - Constructing User Bean and User Service

Step 08 - Implementing GET Methods for User Resource

Step 09 - Implementing POST Method to Construct User Resource

CODE BACKUP FILE: For Reference

Step 10 - Enhancing POST Method to Return Correct HTTP Status Code and Location

Step 11 - Implementing Exception Handling - 404 Resource Not Found

Step 12 - Implementing Generic Exception Handling for all Resources

Step 13 - Exercise: User Post Resource and Exception Handling

Step 14 - Implementing DELETE Method to Delete a User Resource

COURSE UPDATE: Add Dependency spring-boot-starter-validation

Step 15 - Implementing Validations for RESTful Services

COURSE UPDATE: HATEOAS Updates

Step 16 - Implementing HATEOAS for RESTful Services

Step 17 - Overview of Advanced RESTful Service Features

Step 18 - Internationalization for RESTful Services

Step 18 Part 2 - Internationalization for RESTful Services

Step 19 - Content Negotiation - Implementing Support for XML

COURSE UPDATE: Disable XML Format Support

COURSE UPDATE: USE SWAGGER 3.0.0 for Spring Boot 2.2+

Step 20 - Configuring Auto Generation of Swagger Documentation

Step 21 - Introduction to Swagger Documentation Format

Step 22 - Enchancing Swagger Documentation with Custom Annotations

Step 23 - Monitoring APIs with Spring Boot Actuator

Step 24 - Implementing Static Filtering for RESTful Service

Step 25 - Implementing Dynamic Filtering for RESTful Service

Step 26 - Verisoning RESTful Services - Basic Approach with URLs

Step 27 - Versioning RESTful Services - Header and Content Negotiation Approach

Step 28 - Implementing Basic Authentication with Spring Security

Step 29 - Overview of Connecting RESTful Service to JPA

COURSE UPDATE: H2 Database URL

Step 30 - Constructing User Entity and Some Test Data

Step 31 - Updating GET Methods on User Resource to Use JPA

Step 32 - Updating POST and DELETE Methods on User Resorurce to Use JPA

Step 33 - Constructing Post Entity and Many to One Relationship with User Entity

Step 34 - Implementing a GET Service to Retrieve All Posts of a User

Step 35 - Implementing a POST Service to Construct a Post for a User

Step 36 - Richardson Maturity Model

Step 37 - RESTful Web Services - Best Practices

Microservices with Spring Cloud

Section Introduction - Microservices with Spring Cloud

1. Topics:
    1. Spring Cloud Config Server and Bus
    2. Load Balancing with Ribbon and Feign
    3. Implement Naming Server with Eureka
    4. Implementing API Gateway with Zuul
    5. Distributed Tracing with Zipkin
    6. Fault Tolerence with Hystrix
2. Spring cloud components are developed
3. Github repository
    1. https://github.com/in28minutes/getting-started-in-5-steps
        1. Components
        2. Ports
        3. URLs
        4. Complete code
        5. Folder for each component

Step 00 - 01 - Introduction to Microservices

1. Definitions:

    1. Small autonomous services that work together - Sam Newman
    2. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies - James Lewis and Martin Fowler
    3. In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API...contd
    4. These services are built around business capabilities and independently deployable by fully automated deployment machinery...contd

2. Microservices

    1. REST
        1. Usually exposed by REST
    2. & Small Well Chosen Deployable Units
        1. Well thought out boundaries
    3. & Cloud Enabled

1. Building set of small u services instead of one monolith
2. We can have multiple instances of each of the microservices

```
microservice1 -> A1, A2
microservice2 -> B1, B2, B3, B4
microserivce3 -> C1
```

    1. We can bring up another if load is increased

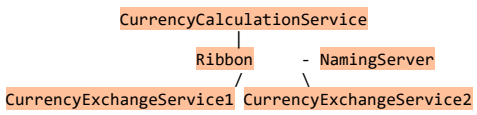
3. Cloud enabling:
    1. Set up an architecture such that it would be able to dynamically adjust and be able to bring new instances up and bring the old instances down

Step 00 - 02 - Challenges with Microservices

1. Challenges with building us:
    1. Bounded context
        1. How to identify boundary?
        2. What to do in each one of them?
        3. What should we not do in each one of them?
        4. We may need business knowledge to establish right boundaries between the us
            1. Experience:
                1. It is an evolutionary process
                    1. Try to follow domain driven design (read the book)
                    2. Need to play around
                    3. Try to identify right boundaries at that point in time
                        1. As we gain more knowledge, we must be able to put in more into the microservices
    2. Configuration management
        1. Say 10 us, 5 envs, 50 instances
            1. Lot of work for operations team
    3. Dynamic scale up and scale down
        1. Establishing technology to be able to do that
            1. Based on load, we may have to change the number of instances automatically
                1. Using dynamic load balancing
                    1. New ones must be used
                2. Must be able to dynamically distributed
    4. Visibility
        1. If there is a bug
            1. How to identify where the bug is
                1. We need centralized logging
                    1. Which microservice caused the problem
            2. We need monitoring
                1. To identify which are down
                2. If there is no enough disk space
    5. Pack of cards
        1. If one fundamental us goes down, the other (pack of cards) us go down
        2. We need fault tolerence in us

Step 00 - 03 - Introduction to Spring Cloud

1. Spring Cloud
    1. Components to solve challenges
    2. Cloud.spring.io
        1. Spring cloud provides tools for developers to quickly build some of the common patterns in distributed systems (e.g. configuration management, service discovery, circuit brakers, intelligent routing, micro-proxy, control bus, one-time tokens, global locks, leadership election, distributed sessions, cluster state). Coordination of distributed systems leads to boiler plate patterns, and using Spring Cloud developers can quickly stand up services and applications that implement those patterns. They will work well in any distributed environment, including the developer's own laptop, bare metal data centres, and managed platforms such as **Cloud Foundary**.
        2. It is not one project
            1. Projects
                1. Spring Cloud Netflix
                    1. Played with us architecture
                        1. Components open sourced
                            1. Eureka, Hystrix, Zuul
                2. Spring Cloud Config
                    1. Centralized config managmenent
                3. Spring Cloud Bus
                    1. Enables us and infrastructure components
                    2. Config server
                    3. API gateway
                4. Finchley M2 reelase of Spring Cloud
    3. Operations
        1. Spring Cloud Config Server
            1. We can store config for each of the server in Git repo (centralized location)
                1. Spring Cloud helps us to expose the centralized config to the us
    4. Dynamic scale up and scale down
        1. Ribbon (load balancing)
            1. Example:



            1. Dynamically distribute load

2. Naming server (Eureka)
    1. All instances of microservices would register with naming server
        1. All us can register
        2. Service discovery
            1. Example: CurrencyCalculationService can ask Eureka naming server -> Give instance of CurrencyExchangeService and Eureka will provide the URL of the service
                1. Establishes dynamic relationship between the services
    2. Ribbon (Client Side Load Balancing)
        1. Example: CurrencyCalculationService will host Ribbon (client)
            1. Load is distributed only among the instances that it gets from Naming Server
    3. Feign (Easier REST clients)
        1. Mechanism to write simple RESTful clients
5. Visibility and Monitoring
    1. Zipkin Distributed Tracing (server)
        1. Spring Cloud Slouth is used to assign ID to request across multiple components
        2. Zipkin Distributed Tracing is used to trace request across multiple components
    2. Netflix [Zuul] API Gateway
        1. uS have common features
            1. Logging
            2. Security
            3. Analytics
            4. ...
6. Fault Tolerance
    1. Hystrix
        1. If services is down, helps to configure default response

Step 00 - 04 - Advantages of Microservices Architecture

1. New technology & process adaption (easily)
    1. Each microservice can be built in different technologies
        1. Can construct us in specific language
    2. We can bring in new processes
2. Dynamic scaling
    1. We can procure hardware or resources dynamically based on the load
3. Faster release cycles
    1. Easier to release
    2. We can bring new features faster to market

Step 00 - 05 - Microservice Components - Standardizing Ports and URL

1. Ports (Github - 03.microservices)
    1. Limits service - 8080, 8081, ...
    2. Spring Cloud Config Server - 8888
    3. Currency Exchange Service - 8000, 8001, 8002, ...
    4. Currency Conversion Service - 8100, 8101, 8102, ...
    5. Netflix Eureka Naming Server - 8761
    6. Netflix Zuul API Gateway Server - 8765
    7. Zipkin Distributed Tracing Server - 9411
2. URLs (same page below ports)

Step 00 - Part 1 - Intro to Limits Microservice and Spring Cloud Config Server

1. Centralized application config or centralized us config
    1. Example:
        1. CurrencyCalculationService
        2. CurrencyExchangeService
        3. LimitsService
    2. Example:
        1. DB config
        2. External service config
        3. Business logic config
        4. ...
2. Example: Currency Exchange Service might have 4 environments
3. Managing config individually for each application and each environment is a difficult task
    1. Solution: Centralization
        1. Put all config in Git
        2. SpringCloudConfigServer takes the config and gives it to the specific us
            1. If LimitsService wants config for dev env of it's service, it can be provided by SpringCloudConfigServer
            2. If CurrencyCalculationService wants config for 3rd instance of it's service, SpringCloudConfigServer can provide it
    2. It is centralized us configuration application
        1. Git repo can be connected to SpringCloudConfigServer
4. Example:

    1. **limits-service-dev.properties**

    ```
    limits-service.minimum=2
    limits-service.maximum=222
    ```

    2. **limits-service-qa.properties**

    ```
    limits-service.minimum=3
    limits-service.maximum=5
    ```

    3. **limits-service.properties**

IMPORTANT: SPRING BOOT AND SPRING CLOUD VERSIONS

1. SNAPSHOT, M1, M2, M3, M4 are work in progress
    1. Spring team is still working on them
    2. Not recommended
    3. Version recommended: 2.3.1+

Step 01 - Part 2 - Setting up Limits Microservice

1. New project:
    1. Newer version of Spring Boot
    2. Group: com.in28minutes.microservices
    3. Artifact: limits-service
    4. Dependencies:
        1. Web
        2. DevTools
        3. Actuator: Monitoring
        4. Config Client: Spring-Cloud-Config-Client
            1. To connect to spring-cloud-config-server
    5. Generate project
    6. Unzip and put it in folder
    7. Import existing maven project in Eclipse
    8. Run the application
        1. Right click run as Java Application

Step 02 - Creation of hard coded limits service

1. We want LimitsService to get config from SpringCloudConfigServer
2. Open **application.properties**

```
spring.application.name=limits-service
```

3. New Controller: LimitsConfigurationController

```
@RestController
public class LimitsConfigurationController {

    @GetMapping("/limits")
    public LimitConfiguration retrieveLimitsFromConfigurations() {
        return new LimitConfiguration(1000, 1);
    }
}
```

4. bean.LimitConfiguration.java

```
public class LimitConfiguration {
    int maximum;
    int minimum;

    protected LimitConfiguration() {

    }

    public LimitConfiguration(int maximum, int minimum) {
        super();
        this.maximum = maximum;
        this.minimum = minimum;
    }

    // Getters and setters
}
```

    1. Open: `localhost:8080/limits`

Step 03 - Enhance limits service to get configuration from application properties

1. application.properties

```
limits-service.minimum=99
limits-service.maximum=9999
```

2. Configuration properties: Configuration.java

```
@Component
@ConfigurationProperties("limits-service") // **(M)**
public class Configuration {
    private int minimum;
    private int maximum;

    // Getters and setters
}
```

3. LimitsConfigurationController.java

```
@Autowired
private Configuration configuration;

...
```

```
        return new LimitConfiguration(configuration.getMaximum(), configuration.getMinimum());
    }
```

Step 04 - Setting up Spring Cloud Config Server

1. New Spring boot project
2. group id: com.in28minutes.microservices
3. artifact id: spring-cloud-config-server
4. search: config server
5. Remove web, ... and keep DevTools and config server
6. Import it into eclipse
7. Dependency: spring-cloud-config-server
8. application.properties

```
spring.application.name=spring-cloud-config-server
server.port=8888
```

Step 05 - Installing Git

1. Install local git
2. download git and install

Step 06 - Creation of Local Git Repository

1. Click Try Git link in Git home page
    1. Type the commands and shows results
2. Open terminal
3. New directory: `mkdir git-localconfig-repo`
4. `cd git-localconfig-repo`
5. `git init`
6. **spring-cloud-config-server**
    1. Right click
    2. Build Path
    3. Link Source
    4. Browse to folder > spring-microservices > 03.microservices > git-localconfig-repo
7. Right click on git-localconfig-repo
    1. New > text file
        1. limits-service.properties

```
limits-service.minimum=8
limits-service.maximum=8888
```

        1. `git add -A`
        2. `git commit -m "first commit"`

Step 07 - Connect Spring Cloud Config Server to Local Git Repository

1. Right click on `git-localconf-repo` - copy the location (local github repo)
2. spring-cloud-config-server > application.properties

```
spring.cloud.config.server.git.uri=file:///<location-of-local-git-repo> # can also give internet address as well
```

3. URL: http://localhost:8888/limits-service/default
    1. Error page:
        1. Need to enable Spring Cloud Config Server

```
@EnableConfigServer // Important!!!
@SpringBootApplication
...
```

            1. Property values get retrieved
                1. Path also
        2. SpringCloudConfigServer is connected to Git
        3. SpringCloudConfigServer has config of LimitsService
    2. We can store config for other services and for multiple environments

Step 08 - Configuration for Multiple Environments in Git Repository

1. DEV, QA, STAGE, PROD environments
    1. `limits-service.properties` - default config
    2. `limits-service-dev.properties` - for dev env

```
limits-service.minimum=1
limits-service.maximum=111
```

3. `limits-service-qa.properties`

```
limits-service.minimum=2
#limits-service.maximum=222 # - picked up from default file
```

4. Commit the files: Allways
    1. Local repository - only then they will be available to SpringCloudConfigServer

```
git add -A
git status
git commit -m "Dev and QA properties"
```

        1. **http://localhost:8888/limits-service/qa**
            1. Properties are in order of priority
                1. QA properties
                2. Default properties
2. Limits-service has logic to take the highest priority values automatically

COURSE UPDATE: Limits Service with >= 2.4.0 of SPRING BOOT

1. Add the following dependency:

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-bootstrap</artifactId>
</dependency>
```

2. Restart the server

Step 09 - Connect Limits Service to Spring Cloud Config Server

1. limits-service:
    1. application.properties - default
    2. To talk to spring-cloud-config-server and pick up the properties from it
        1. Rename the file `application.properties` to:
            1. `bootstrap.properties`

```
spring.application.name=limits-service # very important to pick up values from Git local repo
spring.cloud.config.uri=http://localhost:8888 # connects to spring cloud config service
```

                1. Restart the limits-service
                    1. Picks up default profile
        2. **http://localhost:8080/limits**
            1. Shows properties values

Debugging Problems with Spring Cloud Config Server

1. Debugging us problems can be difficult
2. Troubleshooting guide
    1. Use chrome browser

Step 10 - Configuring Profiles for Limits Service

1. Dev profile and QA profile
    1. Config for limits-service is coming from Git repo
        1. Advantage - config is separated from deployment of the service
2. bootstrap.properties

```
...
spring.profiles.active=dev # picks up properties from limits-services-dev.properties
```

    1. Another way: VM args, Java application args
    2. Open http://localhost:8080/limits
        1. If we change the values, we must commit and only then they are picked up

```
git add *
git commit -m "removed configuration for maximum"
```

            1. Only at startup, the values are picked up so we need to restart limits-service

Step 11 - A review of Spring Cloud Config Server

1. To connect another microservice
    1. Define the configs and commit them to Git
    2. Connect Currency Exchange Service to SpringCloudConfigServer
2. All application config is in one place and it is secure
    1. We have multiple instances of services to be deployed in cloud and it is convenient to put the config in Git
3. Spring Cloud Bus

Step 12 - Introduction to Currency Conversion and Currency Exchange Microservice

1. CurrencyExchangeService
    1. Uses JPA to talk to database and returns exchange value
        1. **http://localhost:8000/currency-exchange/from/EUR/to/INR**
2. CurrencyCalculationService
    1. Uses CurrencyExchangeService to get exchange
        1. **http://localhost:8100/currency-converter/from/USD/to/INR/quantity/1000**

IMPORTANT: Spring Boot And Spring Cloud Versions

1. SNAPSHOT, M1, M2, M3, M4 are work in progress
    1. Spring team is still working on them
        1. They are not recommended
2. Use: 2.3.1+

Step 13 - Setting up Currency Exchange Microservice

1. REST Service for `currency-exchange`
2. Conversion factor
3. Controller: CurrencyExchangeController.java

```
@RestController
public class CurrencyExchangeController {

    @GetMapping("/currency-exchange/from/{from}/to/{to}")
    public ExchangeValue retrieveExchangeValue(@PathVariable String from, @PathVariable String to) {
        return new ExchangeValue(1000L, from, to, BigDecimal.valueOf(65));
    }
}
```

    1. cmd+1
        1. ExchangeValue.java

```
public class ExchangeValue {
    private Long id;
    private String from;
    private String to;
    private BigDecimal conversionMultiple;

    // Generate no args constructor
    // Generate constructor using fields
    // Generate getters and setters
}
```

Step 14 - Creation of simple hard coded currency exchange service

1. CurrencyCalculationService can talk to any of multiple CurrencyExchangeService instances (port is distinguishing here) - typically not a good practice
2. ExchangeValue.java

```
private int port;
...
// Generate getter and setter for port


public class CurrencyExchangeController {

    @Autowired
    private Environment environment;
```

Step 15 - Setting up Dynamic Port in the Response

```
    public ExchangeValue retrieveExchangeValue(...) {
        ExchangeValue exchangeValue = new ExchangeValue(...);
        exchangeValue.setPort(Integer.parseInt(environment.getProperty("local.server.port")));
        return exchangeValue;
    }
```

1. For two instances to run we cannot set in application.properties
2. Right click on project > Run configurations
    1. Duplicate config (right click and duplicate)
        1. Arguments: `-Dserver.port=8001`
            1. Apply
        2. Open: `http://localhost:8001/currency-exchange/from/USD/to/INR`
            1. We can now connect to any of them

Step 16 - Configure JPA and Initialized Data

1. Simple JPA connection to in memory database
2. pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
```

```
</dependency>
```

1. ExchangeValue

```
@Entity
...

    @Id
    private Long id;
    ...
```

3. src/main/resources/data.sql

```
insert into exchange_value (id, from, to, conversion_multiple)
values (10001, 'USD', 'INR', 65);
insert into exchange_value (id, from, to, conversion_multiple)
values (10002, 'EUR', 'INR', 75);
insert into exchange_value (id, from, to, conversion_multiple)
values (10003, 'AUD', 'INR', 50);
```

1. application.properties

```
spring.jpa.show-sql=true
spring.h2.console.enable=true
```

1. Start application - error!!!
    1. from is a keyword in SQL and cannot be used

```
@Column(name = "currency_from")
private String from;

@Column(name = "currency_to")
private String to;
```

1. Table:

```
insert into exchange_value (id, currency_from, currency_to, conversion_multiple, port)
values (10001, 'USD', 'INR', 65, 0);
insert into exchange_value (id, currency_from, currency_to, conversion_multiple, port)
values (10002, 'EUR', 'INR', 75, 0);
insert into exchange_value (id, currency_from, currency_to, conversion_multiple, port)
values (10003, 'AUD', 'INR', 50, 0);
```

1. localhost:8000/h2-console

```
SELECT * FROM EXCHANGE_VALUE;
```

Step 17 - Creation of JPA Repository

1. ExchangeValueRepository.java

```
public interface ExchangeValueRepository extends JpaRepository<ExchangeValue, Long> {
    ExchangeValue findByFromAndTo(String from, String to);
}
```

2. CurrencyExchangeService

```
@Autowired
private ExchangeValueRepository repository;

...
ExchangeValue exchangeValue = repository.findByFromAndTo(from, to);
...
return exchangeValue;
```

1. Check id: 10001
2. Query is shown on Console

Step 18 - Setting up Currency Conversion Microservice

1. New microservice:
    1. com.in28minutes.microservices
    2. currency-conversion-service
    3. Dependencies
        1. Web
        2. DevTools
        3. Actuator
        4. Config Client
    4. Generate project
    5. Import to Eclipse

2. src/main/resources/application.properties

```
spring.application.name=currency-conversion-service
server.port=8100
```

Step 19 - Creation of service for currency conversion

1. http://localhost:8100/currency-converter/from/USD/to/INR/quantity/1000
2. CurrencyConversionController.java

```
@RestController
public class CurrencyConversionController {
    @GetMapping("/currency-converter/from/{from}/to/{to}/quantity/{quantity}")
    public CurrencyConversionBean convertCurrency(@PathVariable String from, @PathVariable String to, @PathVariable BigDecial quantity) {
        return new CurrencyConversionBean(1L, from, to, BigDecimal.ONE, quantity, quantity, 0);
    }
}
```

    1. CurrencyConversionBean

```
CurrencyConversionBean {
    private Long id;
    private String from;
    private String to;
    private BigDecimal conversionMultiple;
    private BigDecimal quantity;
    private totalCalculatedAmount;
    private int port;

    //Generate no arg constructor
    //Generate constructor using fields
    //Generate getters and setters
}
```

        1. Start the application

Step 20 - Invoking Currency Exchange Microservice from Currency Conversion Micro

1. Using RestTemplate

```
public class CurrencyConversionController {
    ...
    Map<String, String> uriVariables = new HashMap<>();
    uriVariables.put("from", from);
    uriVariables.put("to", to);

    ResponseEntity<CurrencyConversionBean> responseEntity = new RestTemplate().getForEntity("http://localhost:8000/currency-exchange/from/{from
    CurrencyConversionBean response = responseEntity.getBody();

    return new CurrencyConversionBean(response.getId(), from, to, response.getConversionMultiple(), quantity, quantity.mutlipy(response.getConv
}
```

    1. Start currency exchange service
    2. Start currency conversion service

Step 21 - Using Feign REST Client for Service Invocation

1. To call REST service (we want code to be simpler than what was written)
    1. Feign makes it easy to invoke other services
    2. Feign provides integration with Ribbon - client side load balancer
2. pom.xml

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud-version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
...
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

    1. Enable feign:

```
@EnableFeignClients("com.in28minutes.microservices.currencyconversionservice")
@SpringBootApplication
public class CurrencyConversionServiceApplication {
    public static void main(String[] args) {
        ...
    }
}
```

        1. Feign proxy needs to be constructed
            1. New interface: CurrencyExchangeServiceProxy.java

```
@FeignClient(name="currency-exchange-service", url="localhost:8000") // from application.properties
public interface CurrencyExchangeServiceProxy {
    @GetMapping("/currency-exchange/from/{from}/to/{to}")
    public CurrencyConversionBean retrieveExchangeValue(@PathVariable("from") String from, @PathVariable("to") String to); // r
}
```

2. New method:

```
@Autowired
private CurrencyExchangeServiceProxy proxy;

@GetMapping("/currency-converter-feign/from/{from}/to/{to}/quantity/{quantity}")
public CurrencyConversionBean convertCurrencyFeign(@PathVariable String from, @PathVariable String to, @PathVariable BigDecimal quan
    CurrencyConversionBean response = proxy.retrieveExchangeValue(from, to);

    return new CurrencyConversionBean(response.getId(), from, to, response. ..., quantity, quantity.multiply(response.getConversionM
}
```

1. We are not concerned about how proxy is connecting or getting details

Step 22 - Setting up client side load balancing with Ribbon

1. Example: Multiple instances exist of "currency-exchange-service"
    1. One instance of "currency-conversion-service" can talk to only one instance of "currency-exchange-service" now
    2. Ribbon: Used to distribute calls
        1. Enable ribbon CurrencyCalculationService
2. CurrencyConversionService

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.3.1.RELEASE</version> <!-- Ribbon is not working with version >= 2.4 -->
    <relativePath/> <!-- lookup parent from repository -->
</parent>

<properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    <java.version>1.8</java.version>
    <spring-cloud.version>Hoxton.SR5</spring-cloud.version>
    <maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>
</properties>
...
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-ribbon</artifactId>
    <version>1.4.7.RELEASE</version>
</dependency>
```

1. Enable ribbon on Proxy:

```
@FeignClient(name="currency-exchange-service")
@RibbonClient(name="currency-exchange-service")
public interface CurrencyExchangeServiceProxy {
    ...
}
```

1. application.properties (of currency-conversion-service)

```
spring.application.name=currency-conversion-service
server.port=8100
currency-exchange-service.ribbon.listOfServers=http://localhost:8000,http://localhost:8001
```

1. Just give list of multiple services

Step 23 - Running client side load balancing with Ribbon

1. Start the applications (currency-exchange-service and currency-conversion-service)
2. `currency-exchange-service`
    1. Start service configured on `8000`
    2. Start service configured on `8001`
        1. Open `Run Configurations`
        2. Right click on `CurrencyExchangeServiceApplication8000`
        3. Duplicate
        4. Arguments

```
-Dserver.port=8002
```

        5. Run
3. Launch `currency-conversion-service`
4. Request: `http://localhost:8100/currency-converter-feign/from/EUR/to/INR/quantity/10000`
    1. Refresh multiple times and port changes

Debugging Problems with Feign and Ribbon

1. It is difficult to debug multiple components
2. Troubleshooting guide: https://github.com/in28minutes/in28minutes-initiatives/tree/master/The-in28Minutes-TroubleshootingGuide-And-FAQ#debugging-problems-with-feign-and-ribbon

Step 24 - Understand the need for a Naming Server

1. Naming Server:
    1. Say we start another instance of currency-exchange-service
        1. We want to increase and decrease dynamically
            1. Naming Server - required - EurekaNamingServer
                1. All instances of microservices register with the naming server (whenever microservice comes up)
        2. When a service wants to talk to another service
            1. It would talk to naming server and ask what are the locations or instances of currency-exchange-service
        3. Important features of naming service
            1. Service registration
            2. Service discovery
                1. microservice asks naming server for details

Step 25 - Setting up Eureka Naming Server

1. Steps:
    1. EurekaNamingServer - component needs to be constructed
    2. CurrencyCalculationService - needs to connect to EurekaNamingServer
    3. CurrencyExchangeService - needs to connect to EurekaNamingServer
    4. Configure Ribbon - to find details from EurekaNamingServer
2. Eureka - offered by Netflix
    1. New project:
        1. groupId: com.in28minutes.microservices
        2. artifactId: netflx-eureka-naming-server
        3. Dependencies: Eureka Server, Config Client (to store config), Actuator, DevTools
        4. Generate Project
        5. Import into Eclipse
3. Enable Eureka Server

```
@SpringBootApplication
@EnableEurekaServer
...
```

    1. application.properties

```
spring.application.name=netflix-eureka-naming-server
server.port=8761
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

        1. Open: `http://localhost:8761`
            1. UI for Eureka:
                1. Shows status
                2. Instances currently registered

Step 26 - Connecting Currency Conversion Microservice to Eureka

1. Connecting Currency Conversion Service and Currency Exchange Service to Eureka server
2. Go to pom.xml of currency-conversion-service

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-eureka</artifactId>
    <version>1.4.7.RELEASE</version>
</dependency>
```

3. CurrencyConversionServiceApplication.java

```
@EnableDiscoveryClient // To register with naming server
...
```

4. application.properties

```
eureka.client.service-url.default-zone=http://localhost:8761/eureka
```

5. Bring up the service and check Eureka dashboard for availability of the service

Step 27 - Connecting Currency Exchange Microservice to Eureka

1. pom.xml of currency-exchange-service

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-eureka</artifactId>
    <version>1.4.7.RELEASE</version>
</dependency>
```

2. CurrencyExchangeServiceApplication.java

```
@EnableDiscoverClient // To register with naming server
...
```

3. application.properties

```
eureka.client.service-url.deault-zone=http://localhost:8761/eureka
```

4. Bring it up and check that two instance of currency exchange service are registered with eureka naming service
5. Problem: we are hardcoding currency exchange service url in currency conversion service
    1. Solution: We want to get the urls from eureka

Course Update: Exclude dependency on jackson-dataformat.xml

1. Open bug with `spring-cloud-starter-eureka`
    1. It uses `jackson-dataformat-xml`
        1. Returns xml responses instead of JSON in browser
            1. Solution: Add exclusion for `jackson-dataformat-xml`

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-eureka</artifactId>
    <exclusions>
        <exclusion>
            <groupId>com.fasterxml.jackson.dataformat</groupId>
            <artifactId>jackson-dataformat-xml</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

Step 28 - Distributing calls using Eureka and Ribbon

1. CurrencyCalculationService wants to get url of CurrencyExchangeService from Eureka
2. Ribbon needs to get instance URL of currency exchange service from naming server
3. CurrencyConversionService
    1. application.properties

```
#currency-exchange-service.ribbon.listOfServers=http://localhost:8000,http://localhost:8001
```

        1. Eureka is already is configured to get the urls
4. Wait for 30 s to 60 s for the entire application to be up and running properly
5. The requests are distributed using round-robbin to currency-exchange-service
    1. If one of the instances goes down, the other instance receives requests

Debugging Problems with Naming Server (Eureka) and Ribbon

1. Step by Step Guide

Step 29 - A Review of Implementing Eureka, Ribbon and Feign

1. Feign - easy to write REST client code
2. Ribbon - load balancer
3. Eureka - naming server (service registration, service discovery)
    1. Ribbon talks to Eureka and gets currency exchange service urls
4. Small microservices can talk to each other

Step 30 - Introduction to API Gateways

1. API Gateways (used to implement common features for all of the microservices)
    1. Authentication, authorization and security
    2. Rate Limits
    3. Fault Tolerance
        1. If a service is not up, a default response needs to be returned
    4. Service Aggregation
        1. If a consumer has to make 15 services for a single process, they can be aggregated into a single call
2. All the calls between microservices go through the API gateway
3. API gateway provides common features
    1. + logging
4. API gateways are good for debugging and doing analytics

Step 31 - Setting up Zuul API Gateway

1. Three steps:
    1. Creation of component for Zuul API Gateway server
    2. What it should do when it gets a request
    3. All requests are configured to pass through the API gateway
2. New Project:
    1. ArtifactId: netflix-zuul-zpi-gateway-server
    2. Dependencies: Zuul, Eureka Discovery (We want to see Zuul instance in Eureka), Actuator, DevTools
3. NetflixZuulApiGatewayServerApplication.java

```
@EnableZuulProxy // Required on Zuul server
@EnableDiscoveryClient
@SpringBootApplication
...
```

4. application.properties (copy from Currency Conversion)

```
apring.application.name=netflix-zuul-api-gateway-server
server.port=8765
eureka.client.server-url.default-zone=http://localhost:8761/eureka
```

Step 32 - Implementing Zuul Logging Filter

1. Logging feature
    1. Any request that goes through gateway is logged
2. ZuulLoggingFilter.java

```
@Component
public class ZuulLoggingFilter extends ZuulFilter {
    private Logger logger = LoggerFactor.getLogger(this.getClass());

    @Override
    public boolean shouldFilter() { // apply filter or not
        return true; // for every request it must be applied
    }

    @Override
    public Object run() { // actual logic
        HttpServletRequest request = RequestContext.getCurrentContext().getRequest();
        logger.info("request -> {} request uri -> {}", request, request.getRequestUri());
        return null;
    }

    @Override
    public String filterType() { // when should filtering happen: before request? (pre) after request? (post) error request? (error)
        return "pre";
    }

    @Override
    public int filterOrder() { // if multiple filters exists, security, logging, ..., set the order here
        return 1;
    }
}
```

    1. Filters can be added for
        1. Security
        2. Rate limiting
        3. ...

Step 33 - Executing a Request Through Zuul API Gateway

1. Lauch API Gateway Server
    1. Pre-requisite (should be running): Eureka, CurrencyExchangeService, CurrencyConversionService
2. Eureka has the other services registered
3. Going through Zuul API gateway:
    1. **http://localhost:8765/{application-name}/{uri}**
        1. http://localhost:8765/currency-exchange-service/currency-exchange/from/EUR/to/INR
            1. Content is logged in Console

Step 34 - Setting up Zuul API Gateway between Microservice Invocations

1. Calls should go through Zuul API Gateway proxy
    1. All services are registered in Eureka
2. CurrencyExchangeServiceProxy

```
@FeignClient(name = "netflix-zuul-api-gateway-server") // talk to Eureka and get the api gateway url
@RibbonClient(name = "currency-exchange-service")
public interface CurrencyExchangeServiceProxy {
    @GetMapping("/currency-exchange-service/currency-exchange/from/{from}/to/{to}")
    public CurrencyConversionBean retrieveExchangeValue(@PathVariable("from") String from, @PathVariable("to") String to);
}
```

    1. http://localhost:8100/currency-converter-feign/from/USD/to/INR/quantity/10
        1. Going through feign
            1. The request gets logged (going through API Gateway)
3. CurrencyConversionService

```
http://localhost:8765/currency-conversion-service/currency-converter-feign/from/USD/to/INR/quantity/10
```

4. Two log entries should appear
    1. One for currency conversion service
    2. One from currency exchange service
5. Helps in central control of logging, security, analytics can be performed, ...

Debugging Problems with Zuul API Gateway

1. Step by Step Guide

Step 35 - Introduction to Distributed Tracing

1. If service is not working fine. We want to debug
    1. How to find out the defect?
        1. Solution: Distributed tracing
            1. We want one place where we can go to see what happened to that particular request
2. There are multiple components
    1. business components
    2. service helpers (Zuul, Eureka, Ribbon, ...)
3. Implementation:
    1. Sleuth + Zipkin
        1. Sleuth: Assigning ID to request (to trace it across component)
        2. Zipkin: Distributed tracing system
            1. All logs from all services are put in message queue (implementation: RabbitMQ)
            2. The messages are sent to Zipkin server where it is consolidated and one can look through different requests and find what happened to a specific request

Step 36 - Implementing Spring Cloud Sleuth

1. Where to use?
    1. One can trace it across components
        1. currency-conversion-service
        2. currency-exchange-service
        3. zuul service
2. Zuul service

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-sleuth</artifactId>
</dependency>
```

    1. NetflixZuulApiGatewayServerApplication.java

```
...
@Bean
public Sampler defaultSampler() { // >= 2.0.0
    return Sampler.ALWAYS_SAMPLER;
}
```

    2. CurrencyConversionService, CurrencyExchangeService
        1. Same configuration
3. Launch applications in correct order
4. Logs in other services

```
// CurrencyExchangeController
public ExchangeValue retrieveExchange() {
    ...
    logger.info("{}", exchangeValue);

// CurrencyConversionController
public CurrencyConversionBean convertCurrencyFeign() {
    ...
    logger.info("{}", response);
```

    1. Sleuth adds id to requests
    2. Run request to currency conversion service
        1. A log with id is generated for currency conversion service
        2. A log with same id is generated for currency exchange service
            1. list of ids (fore each service request)
    3. Problem: Looking into logs of each service is not easy for debugging purposes and connecting the requests is also challenging
        1. Solution: Zipkin - distributed tracing service
            1. Provides centralized dashboard
5. If all applications are slowing down, remove actuator and devtools

Step 37 - Introduction to Distributed Tracing with Zipkin

1. Centralize all logs from all microservices
    1. Has a dashboard
        1. Gives consolidated report
            1. A solution: ELK stack can be used
2. Another solution: Zipkin distributed tracing server
    1. Log message is put in Queue (RabbitMQ)
    2. ZipkinDistributedTracingServer can use database (in-memory is used here)

Step 38 - Installing Rabbit MQ

1. GitHub repo

1. Installation instructions for specific tools are available
    1. RabbitMQ:
        1. Pre-requisites - Erlang
        2. Mac: https://www.rabitmq.com/install-homebrew.html
            1. There is a link for Erlang
            2. `brew update`
            3. `brew install rabbitmq`
2. Launching RabbitMQ:
    1. `/usr/local/sbin/rabbitmq-server`
        1. `~/.bash_profile`

            ```
            export PATH=/usr/local/sbin:$PATH
            ```

        2. `source ~/.bash_profile`
        3. `rabbitmq-server`

Update to Step 39 - Running Zipkin on Windows

1. If you get 404 while downloading jar, use curl command to download

    ```
    curl -sSL https://zipkin.io/quickstart.sh | bash -s
    java -jar zipkin.jar
    ```

    1. https://zipkin.io/pages/quickstart
    2. Run the following command to run Zipkin on Server (Windows)

        ```
        set RABBIT_URI=amqp://localhost
        java -jar zipkin-server-2.7.0-exec.jar
        ```

Step 39 - Setting up Distributed Tracing with Zipkin

1. Services -> RabbitMQ -> ZipkinDistributedTracingServer -> Database
2. Setup (install and connect to RabbitMQ)
    1. New project > Zipkin client was available before but not now
3. Search `zipkin quickstart`
    1. Docker
    2. Java
        1. `wget -O <url>` - copy the url
            1. paste it in browser (may not work)
               2 Download jar file
        2. `java -jar zipkin-server-<version>-exec.jar` (Java 8 is min)
4. Zipkin dashboard:
    1. http://localhost:9411
        1. Find Traces - nothing as of now
        2. Ensure it is running as background service
        3. Configuration: To connect to RabbitMQ (Mac)

            ```
            RABBIT_URI=amqp://localhost java -jar zipkin-server-2.5.2-exec.jar

            # Windows:
            set RABBIT_URI=amqp://localhost
            java -jar zipkin-server-2.5.2-exec.jar
            ```

Step 40 - Connecting Microservices to Zipkin

1. Dependencies: All us pom.xml

    ```
    <!--
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-sleuth-zipkin</artifactId>
    </dependency>
    <!-- AMQP Protocol - Default connection to RabbitMQ -->
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-bus-amqp</artifactId>
    </dependency>
    ```

Updates to Step 40: Use spring-cloud-starter-zipkin and spring-rabbit

1. If Spring Boot >= 2.1.*:
    1. `spring-cloud-starter-zipkin`
    2. `spring-rabbit`
        1. Instead of `spring-cloud-sleuth-zipkin` and `spring-cloud-starter-bus-amqp`

            ```
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-starter-zipkin<artifactId>
            </dependency>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-rabbit</artifactId>
            </dependency>
            ```

Step 41 - Using Zipkin UI Dashboard to Trace Requests

1. Fire a request and check zipkin dashboard
    1. Start all services
        1. CurrencyCalculationService
        2. Zipkin - started from commandline using jar for Spring Boot >= 2.0.0
        3. CurrencyExchangeService
        4. Eureka
        5. Zuul
        6. Ribbon
2. Check Eureka to check if all services are running
3. We can connect Zipkin to Eureka
4. Request: http://localhost:8100/currency-converter-feign/from/EUR/to/INR/quantity/10
5. Open zipkin dashboard: localhost:9411/zipkin
    1. Trace: currency-conversion-service, Find Trace
        1. Click on individual steps to get more details

Debugging Problems with Zipkin

1. Troubleshooting Guide - Step by Step Instructions

Step 42 - Understanding the need for Spring Cloud Bus

1. Problem:
    1. Launch: limit-service, SpringCloudConfigServer
        1. Active profile - qa
        2. Launch another instance of limit-service - 8081
            1. VM args: -Dserver.port=8081
            2. http://localhost:8081/limits
    2. Suppose we change min value to 22
        1. Commit it in git

```
git add *
git commit -m "qa minimum"
```

        2. Refreshing browser page keeps the old values
            1. How to reflect the change?
                1. POST http://localhost:8080/application/refresh
                    1. Full authentication is required error (Spring Boot 2.0.0+ We would get a Resource Not Found Error)
                    2. **Must have `spring-boot-starter-actuator` in limits service**
                        1. Turn off security:
                            1. application.properties

```
management.endpoints.web.exposure.include=* # Enable all actuator URLs
```

            2. Restart applications
                1. Change to 22
                2. Commit the change to git
                3. Run the POST request

```
http://localhost:8080/actuator/refresh (Spring Boot 2.0.0+)
http://localhost:8081/actuator/refresh
```

        3. Suppose 100 instances of limits services are running
            1. Maintenance nightmare (each change to config, we have to call many urls)
            2. Solution: One URL can be used (If hit all are refreshed)

Step 43 - Implementing Spring Cloud Bus

1. Spring Cloud Bus - Kafka, RabbitMQ (AMQP)
    1. Connect both limits-service and SpringCloudConfigServer

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

        1. application.properties

```
management.endpoints.web.exposure.include=*
```

        2. Launch the applications
    2. Upgrade config
        1. change min to 2
        2. Commit the change
        3. **http://localhost:8080/actuator/bus-refresh** (refreshes for all instances)

2. How does it work?
    1. Spring Cloud Bus is run over RabbitMQ
    2. At application startup, all microservice instances register with cloud bus
    3. When there is change to config in any instance, it would send an event over spring cloud bus
    4. Spring cloud bus propagates the event to all the other instances registered with it
3. When amqp dependency is added and saved, spring boot automatically configures connection with RabbitMQ

Step 44 - Fault Tolerance with Hystrix

1. Problem: If any of the microservices are down, then they can bring down the entire dependency chain of microservices

```
CurrencyCalculationService -> CurrencyExchangeService -> LimitsService
```

    1. If LimitsService goes down, the other dependent services may go down as well
2. Fault tolerance:
    1. If a service is not availabe, then dependent service provides default response (graceful behaviour)
    2. Hystrix helps us to build fault tolerance into microservices
3. Dependencies: Limits-Service

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-hystrix</artifactId>
</dependency>
```

    1. LimitsServiceApplication.java

```
@SpringBootApplication
@EnableHystrix
public class LimitsServiceApplication {
    ...
}
```

    2. LimitsConfigurationController.java

```
@GetMapping("/fault-tolerance-example")
@HystrixCommand(fallbackMethod = "fallbackRetrieveConfiguration")
public LimitConfiguration retrieveLimitsFromConfigurations() {
    throw new RuntimeException("Not available"); // some exception is thrown
}

public LimitConfiguration fallbackRetrieveConfiguration() {
    return new LimitConfiguration(999, 9); // some basic behavior is offered in case exception occurs
}
```

        1. Avoids ripple effect due to failure of a single service

Thanks

Bonus Lecture: Our Best Courses

1. Learning paths:
    1. Spring & Spring Boot Developer - https://links.in28minutes.com/udemy-bonus-learning-path-01
    2. Full Stack with React & Angular - https://links.in28minutes.com/udemy-bonus-learning-path-02
    3. Microservices - Docker & Kubernetes - https://links.in28minutes.com/udemy-bonus-learning-path-03
    4. Cloud with AWS, Azure & PCF - https://links.in28minutes.com/udemy-bonus-learning-path-04
    5. AWS with Docker & Kubernetes - https://links.in28minutes.com/udemy-bonus-learning-path-05

Thank You

1. Microservices is a young field and can expect changes
2. Challenges and solutions

Appendix - Introduction to Spring Boot in 10 Steps

0005 - Section Introduction 04 - Introduction to Spring Boot in 10 Steps

Step 1: Introduction to Spring Boot - Goals and Important Features

Step 2: Developing Spring Applications Before Spring Boot

Step 3: Using Spring Initializr to Construct a Project

Step 4: Constructing a Simple REST Controller

Step 5: What is Spring Boot Auto Configuration?

Step 6: Spring Boot vs Spring vs Spring MVC

Step 7: Spring Boot Starter Projects - Starter Web and Starter JPA

Step 8: Overview of Different Spring Boot Starter Projects

Step 9: Spring Boot Actuator

1. pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

    1. Monitoring around the application
        1. We can read metadata
            1. Beans configured
            2. How auto config worked
            3. How many times a specific service is called
            4. How many times a specific service has failed
    2. Actuator exposes REST services which are compliant with HAL standard
    3. We need a hal browser

```
<dependency>
    <groupId>org.springframework.data</groupId>
    <artifactId>spring-data-rest-hal-browser</artifactId>
</dependency>
```

        1. http://localhost:8080/actuator or http://localhost:8080/application or http://localhost:8080 (newer version)
        2. Click links provided
        3. Configuration:

```
management.endpoint.web.exposure.include=* # all management endpoints - performance impact (only enable what is required)
```

            1. Install Spring Tools plugin for auto complete of application.properties
    4. http://localhost:8080
        1. HAL browser
            1. We can browser through actuator: Type /actuator
                1. auditevents
                2. beans - all spring beans configured
                3. /health
                4. /conditions
                    1. configured and not configured beans
                5. heap dump
                6. thread dump
                7. metrics:

```
/actuator/metrics/jvm.memory.used
```

                8. httptrace - all requests executed
                    1. Details of requests
                9. mappings - things mapped to uris

Step 10: Spring Boot Developer Tools

1. Auto restart when changes are made

```
<dependency>
    <groupId>org.springframework.bean</groupId>
    <artifactId>spring-boot-devtools</artifactId>
</dependency>
```

    1. Stop and start takes more time than enabling devtools
        1. Loads only application beans (not beans in maven dependencies)
            1. application relation beans only

Appendix - Introduction to JPA in 10 Steps

Section Introduction - Introduciton to JPA in 10 Steps

Step 1: Object Relational Impedence Mismatch

Step 2: World Before JPA - JDBC, Spring JDBC and myBatis

Step 3: Introduction to JPA

Step 4: Constructing a JPA Project using Spring Initializr

COURSE UPDATE: H2 Database URL

Step 5: Defining a JPA Entity - User

Step 6: Defining a Service to Manage the Entity - UserService and EntityManager

Step 7: Using a Command Line Runner to Save the User to Database

Step 8: Magic of Spring Boot and In Memory Database H2

Step 9: Introduction to Spring Data JPA

Step 10: More JPA Repository: findById and findAll

Docker with Microservices using Spring Boot and Spring Cloud - V2

Step 00 - Match made in Heaven - Docker and Microservices

Step 01 - Installing Docker - Docker

RECOMMENDATION - Use PowerShell in Windows!

Step 02 - Your First Docker Usecase - Deploy a Spring Boot Application

Step 03 - Docker Concepts - Registry, Repository, Tag, Image and Containers

Step 04 - Playing with Docker Images and Containers

Step 05 - Understanding Docker Architecture - Docker Client, Docker Engine

Step 06 - Why is Docker Popular

Step 07 - Playing with Docker Images

Step 08 - Playing with Docker Containers

Step 09 - Playing with Docker Commands - stats, system

Step 10 - Introduction to Distributed Tracing

Step 11 - Launching Zipkin Container using Docker

Step 12 - Connecting Currency Exchange Microservice with Zipkin

Step 13 - Connecting Currency Conversion Microservice & API Gateway with Zipkin

Link for the Next Lecture

Step 14 - Getting Setup with Microservices for Constructing Container Images

Step 15 - Constructing Container Image for Currency Exchange Microservice

Step 16 - Getting Started with Docker Compose - Currency Exchange Microservice

Step 17 - Running Eureka Naming Server with Docker Compose

Step 18 - Running Currency Conversion Microservice with Docker Compose

Step 19 - Running Spring Cloud API Gateway with Docker Compose

Step 20 - Running Zipkin with Docker Compose

Step 21 - Running Zipkin and RabbitMQ with Docker Compose

Kubernetes with Microservices using Docker, Spring Boot and Spring Cloud

Step 00 - Docker, Kubernetes and Microservices - Made for each other

Step 01 - Getting Started with Docker, Kubernetes and Google Kubernetes Engine

Step 02 - Constructing Google Cloud Account

Step 03 - Constructing Kubernetes Cluster with Google Kubernete Engine (GKE)

Step 04 - Review Kubernetes Cluster and Learn Few Fun Facts about Kubernetes

Step 05 - Deploy Your First Spring Boot Application to Kubernetes Cluster

Commands Executed in this Section

Step 06 - Quick Look at Kubernetes Concepts - Pods, Replica Sets and Deployment

Step 07 - Understanding Pods in Kubernetes

Step 08 - Understanding ReplicaSets in Kubernetes

Step 09 - Understanding Deployment in Kubernetes

Step 10 - Quick Review of Kubernetes Concepts - Pods, Replica Sets & Deployments

Step 11 - Understanding Services in Kubernetes

Step 12 - Quick Review of GKE on Google Cloud Console

Step 13 - Understanding Kubernetes Architecture - Master Node and Nodes

Link to Next Lecture

Step 14 - Setup Currency Exchange & Conversion Microservices - Kubernetes

Step 15 - Container Images for Exchange & Currency Conversion Microservices

Step 16 - Deploy Microservices to Kubernetes & Understand Service Discovery

Step 17 - Constructing Declarative Configuration Kubernetes YAML for Microservices

Step 18 - Clean up Kubernetes YAML for Microservices

Step 19 - Enable Logging and Tracing APIs in Google Cloud Platform

Step 20 - Deploying Microservices using Kubernetes YAML Configuration

Step 21 - Playing with Kubernetes Declarative YAML Configuration

Step 22 - Constructing Environment Variables to Enable Microservice Communication

Step 23 - Understanding Centralized Configuration in Kubernetes - Config Maps

Step 24 - Exploring Centralized Logging and Monitoring in GKE

Step 25 - Exploring Microservices Deployments with Kubernetes

Step 26 - Configuring Liveness and Readiness Probes for Microservices with K8S

Step 27 - Autoscaling Microservices with Kubernetes

Step 28 - Delete Kubernetes Cluster and Thank You!